A callsign that changes and changes back

Version 3 — a defect in the shipped segmentation rule, what it costs, and why the fix is not yet the default

EUROCONTROL Performance Review Unit

2026-09-16

Table of contents

Summary 1

1 What v2 shipped, and the question nobody asked 2

2 The defect, found by elimination 3

3 The debounce, and what it costs 3

4 Genuine-merge neutrality 4

5 Against APDF, and the limits of that 5

6 `track_id` again 6

7 Limitations 6

8 Provenance note 7

Summary

The rule OPDI shipped in version 2 breaks a track on a callsign change and never asks whether the callsign changes back. `recommended` (arm A8) groups on the airframe and cuts when the last non-blank callsign genuinely differs from the current one. A *flicker* — a callsign decoded as X, then Y for one sample, then X again — satisfies that test twice: once on the way out, once on the way back. One flight becomes three tracks.

The defect is not rare. Of 44,406 track boundaries `recommended` draws on 2026-06-01, 23.1% — 10,241 boundaries — trace to a flicker's excursion or revert leg, against only 2,199 (5.0%) that are a genuine change. What v2 shipped, and the question nobody asked states the rule; the defect, found by elimination is the measurement.

The fix. `debounced` (arm A9) adds one condition: a new callsign must persist for `callsign_min_persistence_seconds` before the break fires. At a 30 s hold it draws 19.1% fewer tracks than `recommended`. The debounce, and what it costs reports that fragmentation change alongside the guard that tells a suppressed flicker from a genuinely merged flight apart, which is the number that matters more than the raw count.

What it does not change. Genuine-merge neutrality finds the debounce fuses 37 distinct two-flight pairs a day against `recommended`'s 36 — a ratio of 1.03x — and the same rotation pairs appear under both arms, so this is a pre-existing property of the shared gap rules, not something the debounce introduces.

What APDF can and cannot say. Against APDF, and the limits of that is the section most worth reading carefully: block times cannot referee this fix, and the guard that can — movement counts — passes.

What ships. `callsign_min_persistence_seconds` defaults to 0.00, which reproduces `recommended` exactly. `track_id` again states plainly why promoting A9 is not decided on this page.

> **How to read this page**
>
> This is a validation note for one fix, not a new sweep. Each numbered section reports a measurement Tasks 1, 4 or 5 already took; nothing here is computed for the first time on this page. What v2 shipped restates the rule so this page stands alone. The defect is the case for fixing it. The debounce and genuine-merge neutrality are the two ways the fix could have gone wrong, both checked. Against APDF is the external check, with its limits stated before its result. `track_id` again is the operational answer: not yet.
> Every number here is an expression evaluated at render time over a committed CSV — `data/flicker_2026.csv`, `data/fragmentation_2026.csv`, `data/movements_2026.csv` and `data/genuine_merge_2026.csv`. The first three are declared by `benchmarks/regenerate_track_v3.py`, which this page runs before drawing anything; the fourth is a hand-staged follow-up with no declared job of its own. Provenance says which is which and why. Nothing is typed into the prose, which is what stops the text and the tables drifting apart.

1 What v2 shipped, and the question nobody asked

Version 2 replaced the rule behind every `track_id` published up to OPDI v0.0.2 — group on `(icao24, callsign)`, cut on a time gap — with one that groups on `icao24` alone and cuts when the airframe's *last non-blank* callsign genuinely differs from the current one. The configuration key is `segmentation.method`, and it ships as `standard` — the production alias for the arm the segmentation study itself calls `recommended`, arm A8.

The break exists to separate two real flights of the same airframe once their callsigns diverge. It was built and measured against that question, and v2's own evidence — the gain from dropping callsign out of the group key, the small amount of new merging the break claws back — is about whether *the airframe splits when it should*. Nobody asked the complementary question: what happens when the callsign the break fires on turns out not to be a real change at all, but the aircraft's transponder briefly decoding it wrong and then decoding it right again? A8 has no way to tell the two apart. It breaks on the first sample that differs from the last real one it saw, and a flicker satisfies that test on the way out and again on the way back in — twice for one airframe that never changed flight.

2 The defect, found by elimination

The evidence is two independent measurements of the same day, 2026-06-01, agreeing with each other.

By elimination first. `recommended`'s break fires on exactly three conditions: a gap over 30 minutes, a gap over 15 minutes below 5,000 ft, or a genuine callsign change. The two gap rules cannot produce a track boundary with a sub-minute gap — their thresholds are fifteen and thirty minutes — so every sub-minute-gap boundary that day is, by elimination, a callsign break. Classifying every boundary `recommended` actually draws that day by cause:

Table 1: Every track boundary `recommended` draws on 2026-06-01, by cause. 44,406 boundaries total.

Cause	boundaries	share of all boundaries
gap > 30 min	30,961	69.7%
gap > 15 min & < 5,000 ft	1,005	2.3%
flicker revert leg (Y -> X)	5,241	11.8%
flicker excursion leg (X -> Y)	5,000	11.3%
genuine callsign change	2,199	5.0%

Of 12,440 boundaries attributed to a callsign change, 10,241 (23.1% of *every* boundary the rule draws, not just of the callsign-change ones) trace to a flicker's revert or excursion leg. Only 2,199 are a genuine change with nothing reverting either side of it. A flicker does not split a track once — it **trisects** a flight, tripping the break on the way out (excursion leg) and again on the way back (revert leg).

240 boundaries (0.5%) match both a flicker's revert leg and the next flicker's excursion leg at once — an airframe garbling its callsign repeatedly with no stable sample in between. The two flicker buckets summing to the same count either way, that collision count is not, on its own, proof the two are counted disjointly; it is counted rather than assumed.

The raw sequence confirms it directly. The elimination argument infers a flicker from a boundary's shape — sub-minute gap, no altitude signature. The raw callsign sequence, ignoring blanks, says so outright: a value that differs from both its predecessor and its successor, whose predecessor and successor agree with each other (X → Y → X), holds for a median 5 s — the 90th and 99th percentiles are the same 5 s, meaning essentially every flicker is a single sample — and the longest held only 21 s. Nothing that holds for one sample and reverts is a flight changing its callsign; it is a transponder decoding one message wrong.

3 The debounce, and what it costs

`debounced` (arm A9) adds one condition to A8's break: the new callsign must still be in force `callsign_min_persistence_seconds` later. A flicker fails that test — the value has already reverted — and a genuine change passes it, because the new callsign is still there. At `hold = 0` it reproduces `recommended` exactly, which is what the first row of every table on this page is.

Table 2: 2026-06-01, re-segmenting `osn_tracks_clean`. `2+ raw` and `2+ persisted` are per-track callsign counts, not tracks-per-flight.

Arm	tracks	tracks / flight	2+ raw callsigns	2+ PERSISTED callsigns
recommended (A8, ships)	57,689	1.340	47	47
debounced (A9), hold=15s	46,915	1.213	3,066	289
debounced (A9), hold=30s	46,676	1.209	3,056	259
debounced (A9), hold=60s	46,592	1.208	3,053	223

Fragmentation falls. At a 30 s hold the partition drops from 57,689 to 46,676 tracks, 19.1% fewer — close to twice the roughly 5,100 flickers the defect section counted, which is the shape a fix should have: each suppressed flicker removes two boundaries, not one.

`2+ raw callsigns` rises, and that is expected, not a merge. A track that now swallows a flicker legitimately contains both the real callsign and the transient garble, so distinct-raw-callsign count rises with the hold by construction. It cannot, on its own, tell a suppressed flicker from a track that genuinely fused two flights — both look like "more than one callsign in this track."

`2+ PERSISTED callsigns` is the guard, and it does not rise. Counting only callsigns that individually span at least the hold inside a track — a garble spans a sample or two, a genuinely fused second flight spans minutes — `debounced` stays at 289 to 223 across the three holds tested, against `recommended`'s 47 — the same order of magnitude — and it **shrinks as the hold grows**, from 289 at 15 s to 223 at 60 s. A rule that was wrongly fusing distinct flights would show the opposite shape — more persisted-callsign tracks as it gets more permissive. This one shows fewer, which is the signature of a debounce swallowing flicker and nothing else.

> ⚠ What this table cannot measure
>
> `track_fragmentation.py` re-segments `osn_tracks_clean` — state vectors already cleaned under the boundaries `recommended` produced. A candidate arm that would have kept a track together never gets the chance to have it cleaned as one track, so this is a measurement of re-segmenting already-split data, not of segmenting from raw. For counting tracks that is second order; it is not the from-raw measurement. Against APDF reports the one job that is.

4 Genuine-merge neutrality

The guard in the previous section asks whether the debounce *creates* new fused flights. A second, sharper question is whether it changes the count of fusions that were *already there* under `recommended` — the population `2+ persisted callsigns` cannot separate from noise on its own, because a dense pair of distinct callsigns each spanning minutes looks identical whichever arm drew the track.

Table 3: 2026-06-01. A genuine fusion: two callsigns in one track, each spanning >= 300s and >= 100 samples, and string-dissimilar (Levenshtein > 2, not a prefix of each other) – the same floor applied to both arms.

Arm	refined genuine two-flight fusions / day	ratio vs. recommended
recommended (hold=0)	36	1.00x
debounced (hold=30s)	37	1.03x

`debounced` fuses 37 distinct pairs a day against `recommended`'s 36 — 1.03x — and the same RYR/EJU consecutive-rotation pairs appear under **both** arms. The debounce only changes behaviour for callsign excursions that do not persist; two callsigns that are each genuinely present for minutes look identical to A8 and A9 alike, so a rule that fuses those pairs is a property of the shared gap and lookback rules underneath both arms, not something this fix introduces. The residual is pre-existing, and it is not what this page was written to fix.

5 Against APDF, and the limits of that

What the reference can and cannot settle. The obvious check is whether a merged track's block times agree better with APDF's. It is not available. `AOBT` and `AIBT` are frequently rounded to the whole minute, so a difference of tens of seconds measures the reference's quantisation rather than the detector; and ADS-B presence does not share an end with the block times, because a transponder may stay on well past on-block or come on well before off-block. Neither limitation is a defect in APDF — they are what the data is — but together they mean a timing comparison here would produce a number with no interpretation.

What survives both limitations is counting and naming. Movement counts per aerodrome are insensitive to rounding and to transponder dwell. `AP_C_RWY` is categorical and 100% populated at every aerodrome in the study set. Those are what this paper asks of APDF, and they are asked as a **guard** rather than as the result: merging two halves of a cruise-phase split should barely move either, and a large movement shift would mean the change did something other than intended.

The improvement itself is measured internally — fragmentation and the persisted-callsign guard — because those are the quantities the defect actually damages.

Table 4: 2026-06-03. Ratios use OPDI's fragments-removed movement count (`MOVEMENT_DEP/MOVEMENT_ARR`) against APDF's, the same definition track-construction-v2's payoff regression check uses.

Warehouse	departures, OPDI/APDF	arrivals, OPDI/APDF	guard
recommended (opdi-prod, shipped)	1.14x	0.99x	baseline
debounced (opdi/research/a9)	1.01x	1.01x	PASS

The debounced run lands at 1.01x departures and 1.01x arrivals against APDF, with 0 real departures and 0 real arrivals wrongly excluded by the movement rule, and a tighter per-aerodrome

spread than the baseline row. **Guard passed: debouncing does not break movement counting.**

The honest caveat. This is not a clean arm delta. Baseline was built with an earlier vintage of the 04b/movement-rule code than the debounced run below, so the dep_raw/dep_kept change from this row to the debounced row conflates the arm change with the code change and must not be read as the debounce's own effect. The guard this section exists to answer — does debouncing *break* movement counting — is answered no either way; the recommended row's own departure ratio moving from the historical baseline is not evidence for or against the debounce, and is not claimed as any.

What was dropped. A runway-identity check against `AP_C_RWY` was planned and not run: APDF's `SAM ID` cannot join the flight-list's track hash, so there is no key to run the comparison on. It is recorded here as a limitation rather than left silently absent.

6 `track_id` again

> **!** Not decided here
>
> `debounced` defaults **off**. `callsign_min_persistence_seconds` ships at 0.00, which `debounced()`'s own docstring states reproduces `recommended` exactly — the arm is present in the codebase, validated on this page, and not selected by any shipped configuration.

Version 2 already spent the cost of one `track_id` discontinuity: A8 hashes a different group key and suffixes a sample offset instead of a month, so no identifier from before that release joins to any identifier after it. Promoting A9 to the default would spend that cost a second time — not because the group key changes again (`debounced` groups on `icao24`, exactly as `recommended` does), but because `id_from_start_time` means a track's identifier is derived from its own first sample, and suppressing a flicker changes which sample that is for every track a flicker used to cut in two. The count of published identifiers surviving a promotion would not be zero the way v2's was, but it would not be the identity map either, and nothing on this page measured how far from it.

That is a decision for the contract owner, weighing a real fragmentation defect against a second continuity break on a published contract, not a result this validation page can produce by itself. It is not taken here.

7 Limitations

One day, mostly. The boundary-cause breakdown, the flicker holding-time distribution and the fragmentation table are all 2026-06-01. The APDF movement guard is 2026-06-03. Nothing on this page claims the day is representative; it is the day Tasks 1, 4 and 5 measured and the day this page reports.

Re-segmentation, not re-ingestion, for fragmentation. The debounce section states this in place: `track_fragmentation.py` re-segments already-cleaned state vectors, whose cleaning was itself partitioned by `recommended`'s own boundaries. The APDF guard is the one job here that runs a full pipeline — `run_debounced_day.py`, steps 02 through 04b — from raw.

The genuine-merge comparison is a hand-run follow-up, not a declared job. Genuine-merge neutrality reads `debounced_vs_recommended_merge.py`'s output, which Task 4 ran once

to interpret the persisted-callsign guard's residual. It is not wired into `regenerate_track_v3.py` — see Provenance for why and what re-declaring it would take.

Runway identity was dropped, not measured and found equal. Against APDF states why: there is no join key between APDF's `SAM ID` and the flight list's track hash.

No downstream milestone comparison. `debounced` was not scored on step 04 events or measurements, only on the partition itself and on the flight list's movement counts. Anything that consumes a track more granularly than "how many movements landed at this aerodrome" is untested here.

8 Provenance note

`flicker_2026.csv`, `fragmentation_2026.csv` and `movements_2026.csv` are each declared as one job in `benchmarks/regenerate_track_v3.py` (`flicker_2026`, `fragmentation_2026`, `movements_2026`). All three are staged here as **hand-transcribed committed values**, not as output of a run this checkout performed: Tasks 1, 4 and 5 ran `callsign_flicker.py`, `track_fragmentation.py` and `movement_counts.py` on the cluster at commit `da2dcde` (`bench: debounce validation scripts (fragmentation residual + APDF movement guard)`), and the numbers were recorded from those runs rather than regenerated by this page. `--out-name` support was added to all three scripts afterward, in the same change that added `regenerate_track_v3.py` itself — so the code fingerprint the entrypoint checks against has moved on from the commit that actually produced these figures, and `--check` correctly reports all three as stale. Re-running them is what clears that, and would write files this page expects to be identical to the ones committed here, because nothing in the classification logic changed — only the ability to write a CSV. This page renders with `OPDI_RENDER=skip` and reports these three outputs as staged rather than paying a cluster session for a number that would not move.

`genuine_merge_2026.csv` has no declared job at all. It is `debounced_vs_recommended_merge.py`'s output, a follow-up Task 4 ran once, by hand, to interpret `fragmentation_2026.csv`'s persisted-callsign guard column once that guard showed a nonzero residual. The module docstring in `regenerate_track_v3.py` records what re-declaring it as a job would need: the script is read-only and writes nothing a staleness check can key on today, so wiring it in means giving it the same `--out-name`/`--results-dir` treatment the other three received, not just adding a `Job` entry.

Every number in this file traces to the working notes for this fix — `task-6-numbers.md`, under `.superpowers/sdd/` in the `opdi` repository, dated 2026-09-15 — which is itself a transcription of Tasks 1, 4 and 5's cluster output. That file, not this one, is the record of how each number was actually computed; this page's job is to read it consistently, not to re-derive it.